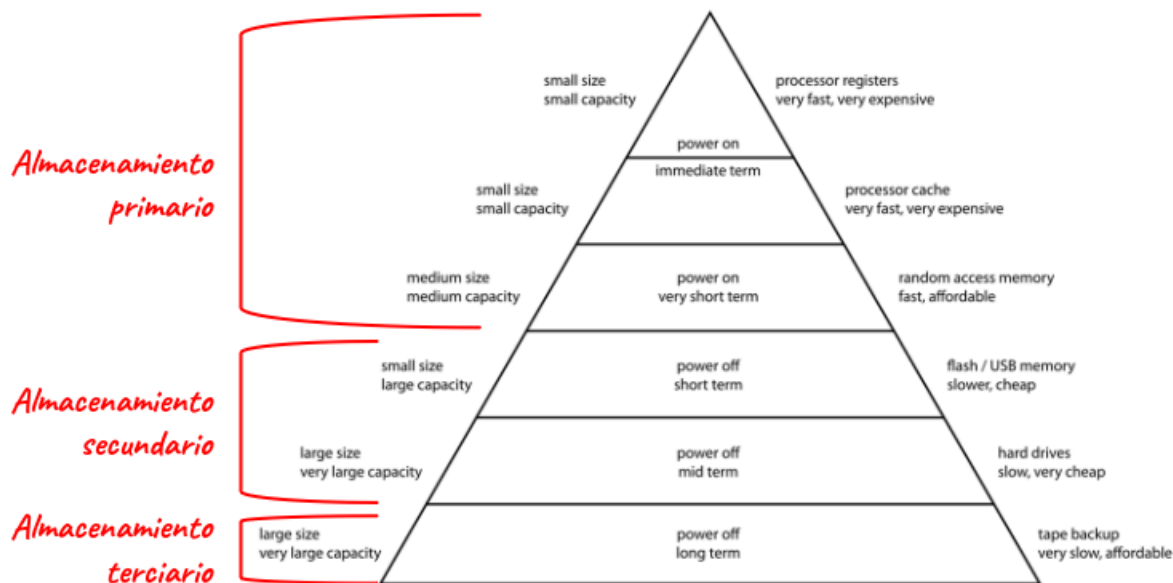


T7.3 - Almacenamiento e Índices

Almacenamiento

Tipos de almacenamiento



Discos Magnéticos

En discos magnéticos, el almacenamiento se organiza en **bloques**.

- Son de *longitud fija* (4KB - 16KB)
- Se *manipulan de forma atómica* (se leen y se escriben por completo)
- Lectura de un bloque $\Rightarrow$ copiar *todo* su contenido a la memoria.

El **acceso secuencial** implica *leer bloques físicamente pegados* uno al lado del otro.

El **acceso aleatorio** se da cuando se necesitan *bloques que están físicamente en cualquier lugar* del disco.

Sería lo ideal tener todos los bloques juntos, pero en la práctica esto no pasa: ya sea por inserciones de bloques nuevos donde haya lugar o borrados que crean espacios.

Hay tres técnicas de optimización:

- **Buffering:** *almacenar bloques ya leídos en memoria* para acelerar futuras lecturas y no tener que ir a buscarlo de nuevo al disco.

- **Read-ahead:** *almacenar bloques consecutivos en memoria* incluso si todavía no se pidieron.
- **Scheduling:** *ordenar pedidos de bloques* según su ubicación física para acelerar el proceso. Esto minimiza movimientos del brazo del disco.

Organización de archivos - Registros

Registro → una *tupla de una relación* cualquiera *guardada en disco*.

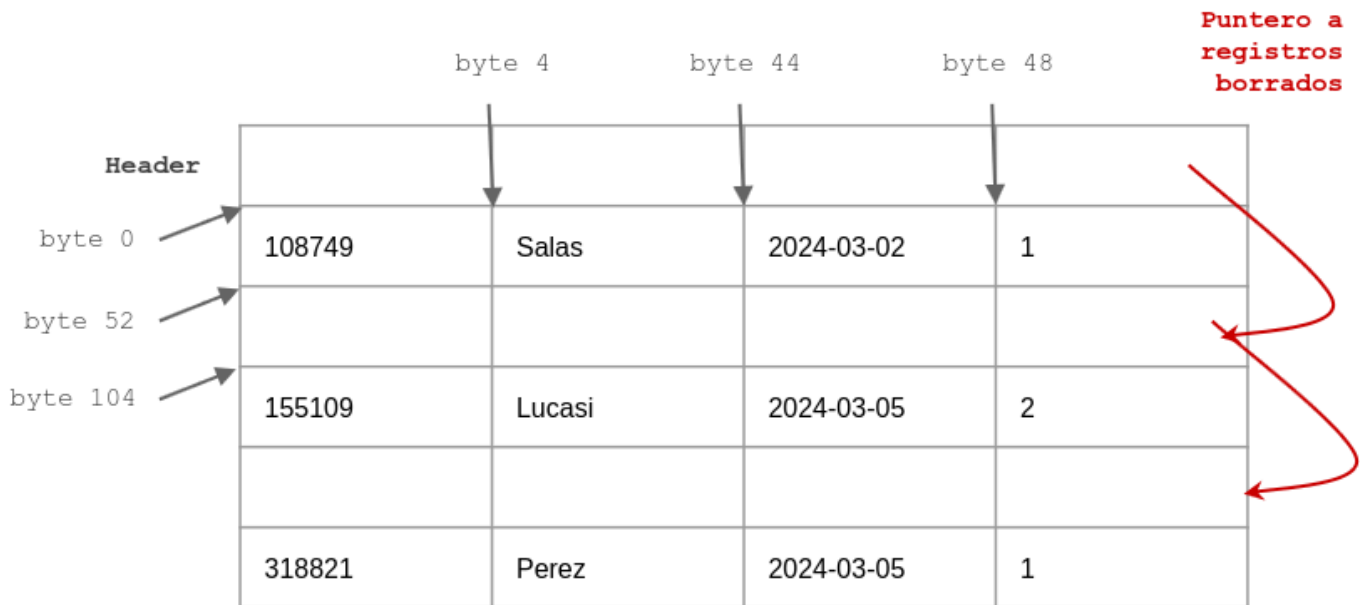
Sirven para *organizar el almacenamiento de los datos* → qué bytes corresponden a qué atributo, en qué posición...

No deben superar el tamaño del bloque ni *sobrepasar sus fronteras*. Pueden ser de tamaño fijo o variable.

Registros de tamaño fijo

Facilita el almacenamiento, pero *impide tipos de dato de tamaño variable* como **VARCHAR**. Cuando se borran este tipo de registros, se dejan *espacios libres que se pueden volver a llenar* con otros registros.

Hay *headers a nivel de bloque*, que tienen punteros que apuntan a dónde hay lugares disponibles (registros borrados).



Registros de tamaño variable

Permite *dominios de distinto tamaño* sin necesidad de reservar ese espacio.

Permite *almacenar más de una relación en el mismo archivo*. Un archivo puede tener bloques con registros de **Empleados** mezclados con registros de **Departamentos**.

Tiene *más dificultades para encontrar valores específicos* de un atributo y *dónde* está cada registro, porque los mismos pueden tener distinto tamaño.

Hay *headers a nivel bloque y a nivel registro*.

Cada bloque tiene un header con información de sus registros: cantidad total de registros que hay en ese bloque, una lista con las ubicaciones de cada registro, y la ubicación de los espacios vacíos en el bloque.

Metadata

Para poder operar con los datos en el disco, el motor tiene que mantener **metadatos**, que son:

- Nombres de relaciones, atributos y sus tipos.
- Restricciones de dominio
- Índices

Índices

Índices y optimización

Para que el motor de una base de datos resuelva la consulta

```
SELECT Name, Composer
FROM Track
WHERE TrackId = 1752
```

SQL

debe:

1. Hacer un **full scan**: se piden todos los registros de **Track** y se carga a memoria.
2. Fijarse qué valor tiene el atributo **TrackId**.
3. Si cumple con la condición del **WHERE**, o sea si **TrackId = 1752**, se agrega al resultado.

Un **índice** es una *estructura de datos* orientada para *optimización de búsquedas* por algún valor específico o rango de valores.

Tener un índice implica:

- Saber *dónde almacenarlo* (costo adicional en disco y memoria).
- *Cómo actualizar el índice* cada vez que haya una escritura, borrado o actualización de algún registro (otro costo adicional).

Estos se construyen sobre una **clave**, que *puede ser un atributo* o un *conjunto de atributos sobre el cual se quiere buscar*.

Si se hacen consultas frecuentes mirando el apellido (**WHERE apellido = "Elosegui"**), conviene crear un índice cuya clave sea **apellido**.

Clave no es lo mismo que clave candidata. Una clave puede ser cualquier atributo, tenga o no valores únicos. Se puede tener un índice sobre los apellidos, por más que dos personas sí puedan tener el mismo apellido.

Vamos a ver tres tipos de índices:

1. **Clustering index:** los *registros* en disco están *físicamente guardados en el orden* que *indica la clave del índice*. En el caso de **apellido**, los registros van a estar ordenados alfabéticamente (en este caso, por ser tipo **string**).

```
Bloque 1: Lucasi, Marrone  
Bloque 2: Pérez, Rocca  
Bloque 3: Salas
```

Se reordenan los registros dentro de los bloques para que sigan el orden de la clave. Para que los bloques se puedan recorrer en orden sin estar físicamente pegados, *se conectan con punteros*.

Estos índices pueden ser *densos*, que apuntan a todos los registros; o *ralos*, que aplican sólo a algunos registros.

2. **Secondary index:** los *registros* en disco *quedan donde están*, sin importar el orden.

```
Bloque 1: Salas, Marrone  
Bloque 2: Lucasi, Rocca  
Bloque 3: Pérez
```

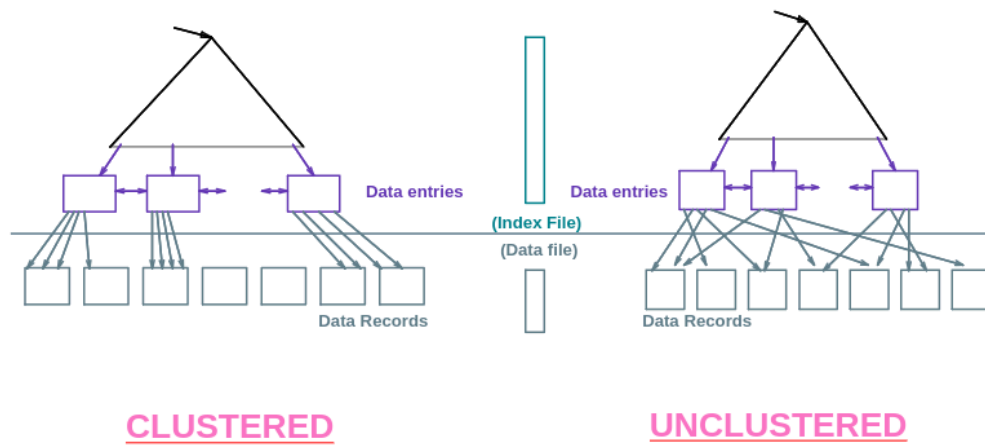
Están *desordenados*, en el *orden que se insertaron*.

Secondary index es una estructura aparte que dice:

```
Lucasi → Bloque 2, posición 1  
Marrone → Bloque 1, posición 2  
Pérez → Bloque 3, posición 1  
Rocca → Bloque 2, posición 2  
Salas → Bloque 1, posición 1
```

Para buscar **apellido = "Pérez"**, el motor va al índice, encuentra la entrada, y va a donde apunte el puntero.

Estos índices son siempre densos.



El gráfico de la izquierda muestra el **clustering index**: arriba está el *index file* con el árbol y las *data entries*. Abajo están los *data records* (los registros reales en disco). La primera hoja apunta a los primeros bloques, la segunda a los siguientes, etc. Los registros están ordenados igual que el índice.

El gráfico de la derecha muestra el **secondary index**: el índice arriba es igual, pero las flechas van para todos lados, cruzándose entre sí. *Cada entrada del índice* apunta a un registro que *puede estar en cualquier bloque*. Los registros no siguen el orden del índice.

La estructura sobre la cual se construyen los índices importa: pueden ser **ordenados** si se apoyan sobre estructuras de ordenamiento, con complejidad de lectura y escritura de $O(\log n)$; o bien pueden ser **de hashes**, que se basan en estructuras de tabla de hash con acceso en $O(1)$, pero no sirven para búsquedas de rango.

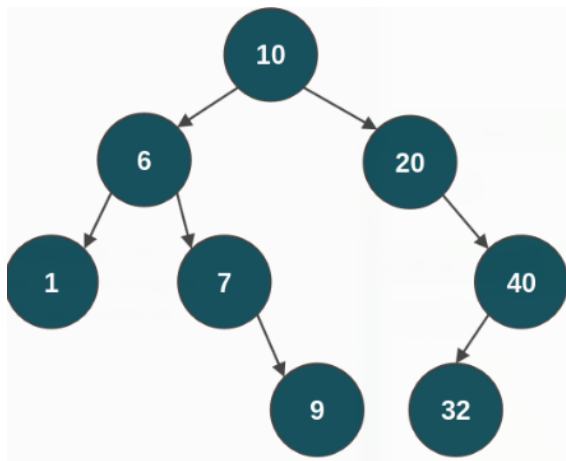
Árboles de búsqueda

Árboles de búsqueda binaria

Cada nodo tiene como máximo dos hijos.

Para todo nodo:

- Los elementos del *subárbol izquierdo* son todos *menores* al valor del nodo raíz.
- Los elementos del *subárbol derecho* son todos *mayores* al valor del nodo raíz.



Una búsqueda en un árbol como estos será tan cara como la altura del árbol: $O(\log n)$ si está balanceado, y en caso de no estarlo, será $O(n)$.

Árboles AVL

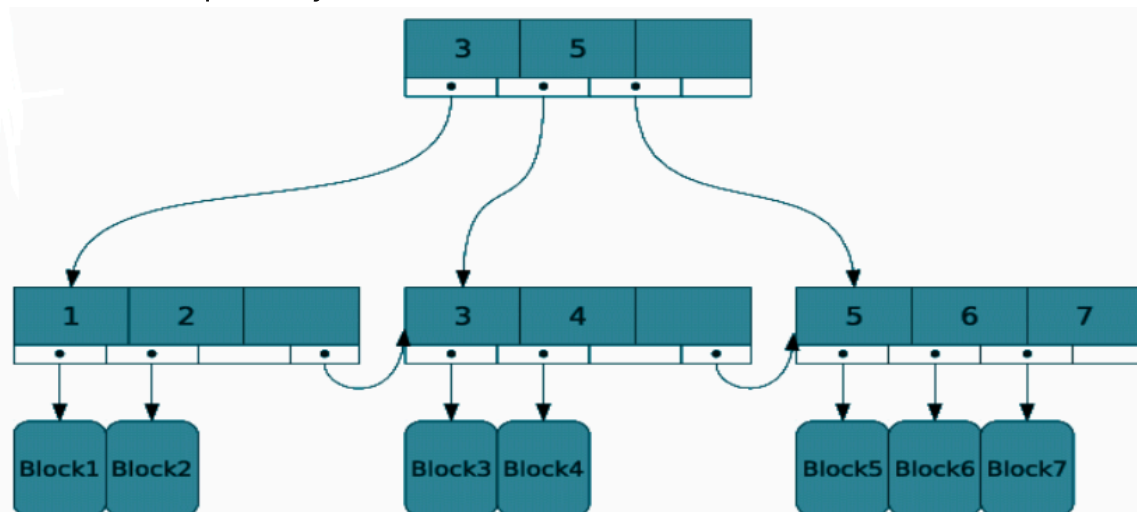
Es un *árbol de búsqueda auto balanceante*.

Cumple las propiedades:

- Todos los nodos tienen un valor único.
- La altura entre los dos subárboles hijos de un nodo puede diferir como máximo en 1.
- Cada inserción o borrado puede ocasionar un rebalanceo.
- El rebalanceo implica una o más operaciones de rotación entre nodos.

Árboles B+

- Cada uno de los nodos tiene hasta b hijos.
- Las hojas son las únicas que almacenan valores.
- Las hojas están conectadas a modo de lista enlazada.
- Son de baja altura.
- Permiten búsquedas y recorrer en forma secuencial.



Nivel de hojas → Bloques, donde están los registros reales.

Nivel intermedio → hay tres nodos: $[1,2]$, $[3,4]$, $[5,6,7]$.

Cada valor es una clave que apunta hacia abajo a un bloque.

El valor 1 apunta a Block1, 2 apunta a Block2...

Estos nodos están conectados entre sí.

Nivel raíz → hay dos nodos "guías", 3 y 5.

Si se busca algo menor a 3, se baja a la izquierda al nodo $[1,2]$.

Si se busca algo entre 3 y 5, se baja por el medio al nodo $[3, 4]$.

Si se busca algo mayor o igual a 5, se baja por la derecha al nodo $[5, 6, 7]$.

La búsqueda de un valor puntual, el insert y el delete tendrán complejidad $O(\log_b n)$, siendo b la cantidad de valores almacenados.